Machine Learning Techniques

BCS-055 · AKTU Exam Answers · Topper answer-sheet style

> **How to use these notes:** ML numericals (ID3, Naive Bayes, Candidate Elimination, K-Means, Backpropagation) come up **every year**. Write the **formula, then the substitution, then the boxed answer**. Each one is solved step by step below, inside its unit.

UNIT 1: INTRODUCTION & LEARNING SYSTEMS

Section A: 2-Mark Questions

Q1. Define a well-posed learning problem. *[2 Marks]*

Ans. Tom Mitchell: *"A program learns from **experience E** with respect to **task T** and **performance measure P** if its performance at T, measured by P, improves with E."* A problem is **well-posed** when T, P and E are all defined. Example (checkers): **T** = play checkers, **P** = % of games won, **E** = games played against itself.

Q2. ML vs. Deep Learning; Occam's Razor. *[2 Marks]*

Ans.

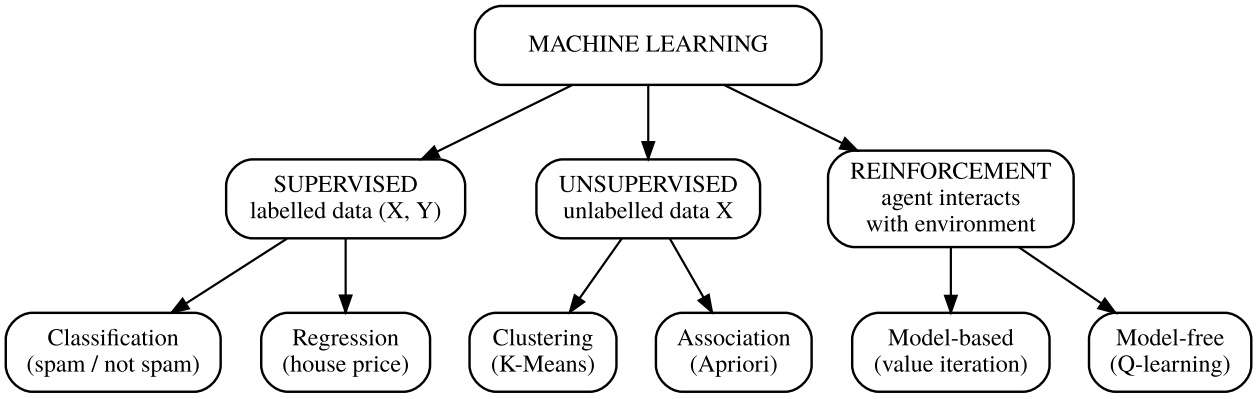
Machine Learning	Deep Learning
Manual feature extraction	**Automatic** feature learning
Works on small data, runs on a CPU	Needs **big data** and a **GPU**
Interpretable (decision tree)	Black box (CNN, RNN)

Occam's Razor: *prefer the **simplest hypothesis** that fits the data*. Simpler models **generalize better** and overfit less. For example, ID3 prefers shorter trees.

Section B: 7-Mark Questions

Q3. Compare supervised, unsupervised and reinforcement learning with examples. *[7 Marks]*

Ans. The three paradigms differ in the **kind of feedback** the learner gets.



The three learning paradigms

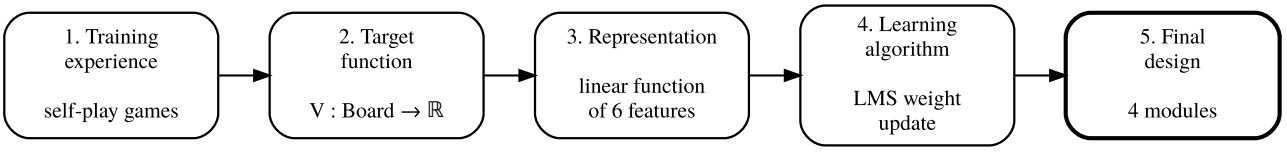
1. **Supervised:**
   - Learns from **labelled data (X, Y)** a function $f: X \to Y$.
   - **Classification** and **regression**.
   - Algorithms: SVM, Decision Tree, k-NN.
   - Example: **spam filtering**, house-price prediction.
2. **Unsupervised:**
   - **Unlabelled data**; it finds hidden structure.
   - **Clustering**, association, PCA.
   - Algorithms: K-Means, Apriori.
   - Example: **customer segmentation**, market-basket analysis.
3. **Reinforcement:**
   - An **agent** acts in an **environment** and gets **rewards**. It learns a **policy** by trial and error.
   - Algorithms: Q-learning, DQN.
   - Example: **AlphaGo**, self-driving cars.

Basis	Supervised	Unsupervised	Reinforcement
Data	Labelled	Unlabelled	Interaction
Feedback	Correct answer	None	**Delayed reward**
Goal	Predict outputs	Find patterns	Maximize total reward
Example	Spam filter	Customer segments	Game playing

Conclusion: ChatGPT uses all three: supervised training, self-supervised learning, and **RLHF**.

Q4. Design a learning system: the checkers case study. *[7 Marks]*

Ans. Designing a learner means choosing the **training experience, the target function, its representation and the learning algorithm**.



Design choices for the checkers learner (the option chosen at each step)

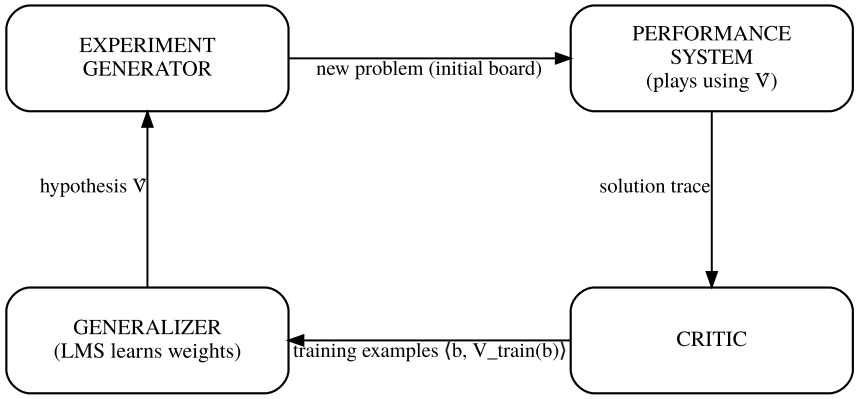
1. **Training experience:** the program plays **games against itself**. This gives **indirect feedback** (only the final win or loss), which raises the credit-assignment problem.
2. **Target function:** V: Board $\rightarrow \mathbb{R}$, where V = +100 for a win, −100 for a loss and 0 for a draw.
3. **Representation:** a linear combination of 6 features:

$$\widehat{V}(b) = w_0 + w_1 x_1 + \cdots + w_6 x_6$$

Here x_1, x_2 = black and red pieces, x_3, x_4 = black and red kings, x_5, x_6 = pieces threatened. 4. **Learning algorithm (LMS):** estimate the training value as $V_{train}(b) \leftarrow \widehat{V}(\text{Successor}(b))$, then update each weight:

$$w_i \leftarrow w_i + \eta \left(V_{train}(b) - \widehat{V}(b)\right) x_i$$

5. **Final design** (four modules in a loop):



Final design: four modules working in a loop

Conclusion: this framework led to Samuel's checkers player and the ideas behind AlphaZero.

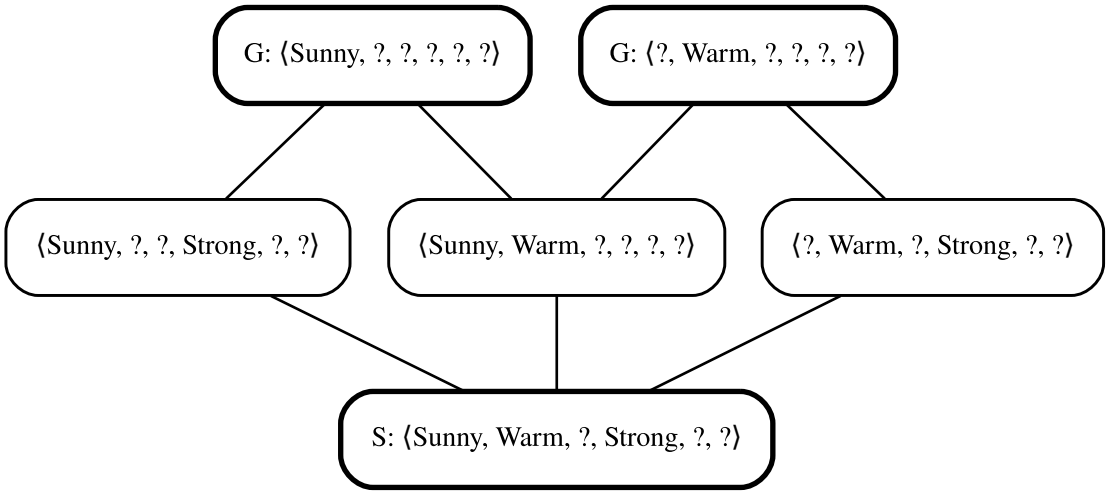
Q5. Trace the Candidate Elimination algorithm (EnjoySport). *[7 Marks]*

Ans. Candidate Elimination finds the **version space** using two boundaries. **S** is the most specific boundary and **G** is the most general. **A positive example generalizes S; a negative example specializes G.**

Ex	Sky	Temp	Humid	Wind	Water	Forecast	Sport
1	Sunny	Warm	Normal	Strong	Warm	Same	**Yes**
2	Sunny	Warm	High	Strong	Warm	Same	**Yes**
3	Rainy	Cold	High	Strong	Warm	Change	**No**
4	Sunny	Warm	High	Strong	Cool	Change	**Yes**

Step	S	G
0	⟨ø,ø,ø,ø,ø,ø⟩	⟨?,?,?,?,?,?⟩
1 (+)	⟨Sunny, Warm, Normal, Strong, Warm, Same⟩	⟨?,?,?,?,?,?⟩
2 (+)	⟨Sunny, Warm, **?**, Strong, Warm, Same⟩	⟨?,?,?,?,?,?⟩
3 (−)	unchanged	⟨Sunny,?,?,?,?,?⟩, ⟨?,Warm,?,?,?,?⟩, ⟨?,?,?,?,?,Same⟩
4 (+)	⟨Sunny, Warm, ?, Strong, **?, ?**⟩	⟨Sunny,?,?,?,?,?⟩, ⟨?,Warm,?,?,?,?⟩

*At step 3, only specializations that are **more general than S** are kept. At step 4, ⟨?,?,?,?,?,Same⟩ is **removed** because it does not cover the positive example (Forecast = Change).*



Final version space: the S boundary, the three hypotheses between, and the G boundary

S = ⟨Sunny, Warm, ?, Strong, ?, ?⟩ G = {⟨Sunny,?,?,?,?,?⟩, ⟨?,Warm,?,?,?,?⟩}

UNIT 2: REGRESSION, BAYESIAN LEARNING & SVM

Section A: 2-Mark Questions

Q1. Maximum Likelihood hypothesis; two uses of logistic regression. *[2 Marks]*

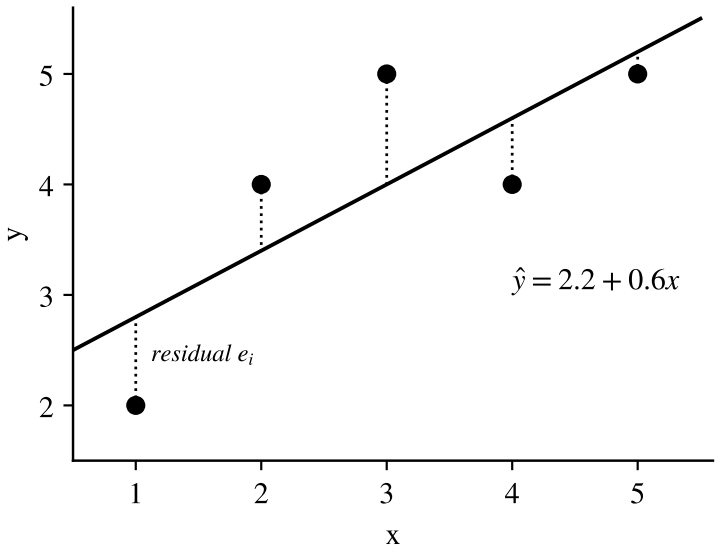
Ans.

- $h_{ML} = \arg\max_{h \in H} P(D \mid h)$: the hypothesis that makes the observed data **most probable**. It is the same as MAP when all hypotheses are equally likely.
- **Logistic regression uses:** (1) **disease prediction** (Yes/No), (2) **spam detection**. Also fraud detection and churn prediction.

Section B: 7-Mark Questions

Q2. Derive the least-squares solution for linear regression. *[7 Marks]*

Ans. For the line $\hat{y} = \beta_0 + \beta_1 x$, **least squares** minimizes $SSE = \sum \left(y_i - \beta_0 - \beta_1 x_i\right)^2$.



Least-squares line through the five data points; each residual is the vertical gap between a point and the line

Step 1:

$$\frac{\partial SSE}{\partial \beta_0} = -2\sum \left(y_i - \beta_0 - \beta_1 x_i\right) = 0 \;\Rightarrow\; \beta_0 = \bar{y} - \beta_1 \bar{x}$$

Step 2:

$$\frac{\partial SSE}{\partial \beta_1} = -2\sum x_i\left(y_i - \beta_0 - \beta_1 x_i\right) = 0 \;\Rightarrow\; \sum x_i y_i = \beta_0 \sum x_i + \beta_1 \sum x_i^2$$

Step 3: substitute β_0 and simplify:

$$\beta_1 = \frac{\sum (x_i - \bar{x})\left(y_i - \bar{y}\right)}{\sum (x_i - \bar{x})^2}, \qquad \beta_0 = \bar{y} - \beta_1 \bar{x} \qquad \text{Matrix form: } \beta = \left(X^T X\right)^{-1} X^T y$$

Example: x = 1, 2, 3, 4, 5 and y = 2, 4, 5, 4, 5. Here $\bar{x} = 3$, $\bar{y} = 4$, $\sum (x - \bar{x})\left(y - \bar{y}\right) = 6$ and $\sum (x - \bar{x})^2 = 10$.

$$\beta_1 = 0.6, \qquad \beta_0 = 4 - 1.8 = 2.2 \qquad \Rightarrow \qquad \hat{y} = 2.2 + 0.6x$$

Conclusion: used for sales forecasting and price prediction. It is sensitive to outliers.

Q3. Naive Bayes classifier: classify X = (Confident = Yes, Studied = Yes, Sick = No). *[7 Marks]*

Ans. Naive Bayes uses Bayes' theorem and assumes the **features are independent given the class**:

$$P(C \mid X) \propto P(C) \prod_i P(x_i \mid C)$$

No.	Confident	Studied	Sick	Result
1	Yes	No	No	Fail
2	Yes	No	Yes	Pass
3	No	Yes	Yes	Fail
4	No	Yes	No	Pass
5	Yes	Yes	Yes	Pass

Step 1: Priors. $P(\text{Pass}) = \frac{3}{5}$ and $P(\text{Fail}) = \frac{2}{5}$.

Step 2: Likelihoods.

	Pass (3 rows)	Fail (2 rows)
Confident = Yes	2/3	1/2
Studied = Yes	2/3	1/2
Sick = No	1/3	1/2

Step 3: Multiply.

$$\text{Pass: } \frac{3}{5} \cdot \frac{2}{3} \cdot \frac{2}{3} \cdot \frac{1}{3} = 0.0889 \qquad \text{Fail: } \frac{2}{5} \cdot \frac{1}{2} \cdot \frac{1}{2} \cdot \frac{1}{2} = 0.0500$$

Step 4: Posterior.

$$P(\text{Pass} \mid X) = \frac{0.0889}{0.1389} = 0.64$$

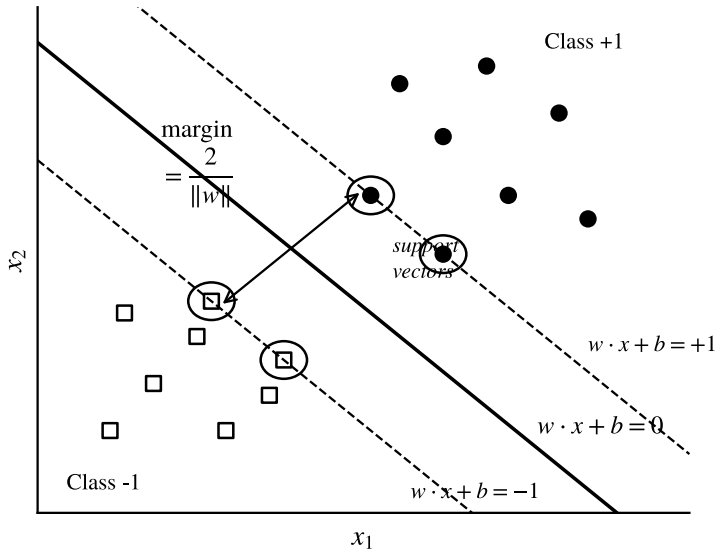
Result=**PASS**

If only (Confident, Sick) is given: Pass = 0.6 × 2/3 × 1/3 = 0.133, Fail = 0.4 × 1/2 × 1/2 = 0.100, so the answer is still **PASS**.

> **Exam Tip:** if any count is 0, use Laplace smoothing: $\frac{n+1}{N+k}$.

Q4. Discuss the SVM: architecture, margin and core issues. *[7 Marks]*

Ans. An **SVM** finds the **hyperplane** $w \cdot x + b = 0$ that separates the classes with the **maximum margin**. The closest points are the **support vectors**.



Maximum-margin hyperplane with support vectors (circled)

1. **Margin** $= \frac{2}{\| w \|}$, so maximizing the margin means minimizing $\| w \|$.
2. **Hard margin:**

$$\min \tfrac{1}{2} \| w \|^2 \quad \text{s.t.} \quad y_i (w \cdot x_i + b) \geq 1$$

3. **Soft margin:** add slack variables ξ_i and a penalty $C \sum \xi_i$. A large C gives a narrow margin.
4. **Kernel trick:** $K\left(x_i, x_j\right) = \phi (x_i) \cdot \phi \left(x_j \right)$ handles non-linear data. Kernels: linear, polynomial, **RBF**.
5. **Core issues:**
   - Choice of kernel, and tuning **C and** γ.
   - **Slow** on big data ($O\left(n^2\right)$ to $O\left(n^3\right)$).
   - Needs **feature scaling**.
   - Binary only (use OvO / OvR for more classes).
   - No probability output.

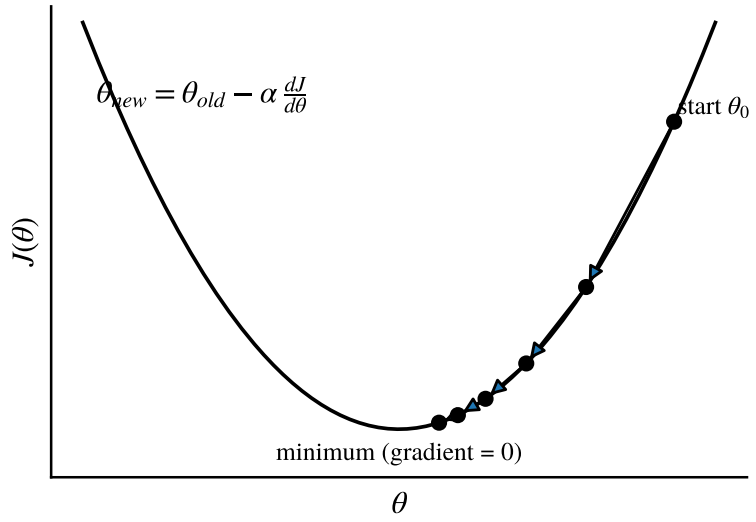
Hard margin	Soft margin
No errors allowed	Errors allowed (slack ξ)
Sensitive to outliers	Robust

Conclusion: used for text classification, face detection and handwriting recognition.

Q5. Explain Gradient Descent. *[7 Marks]*

Ans. Gradient descent is an **iterative** method that minimizes a cost $J (\theta)$ by moving **against the gradient**:

$$\theta_j := \theta_j - \alpha \frac{\partial J}{\partial \theta_j}, \qquad J (\theta) = \frac{1}{2m} \sum (h_\theta (x_i) - y_i)^2$$



Gradient descent steps down the cost curve towards the minimum; steps get smaller as the slope flattens

1. The gradient points **uphill**, so subtracting it moves θ **downhill**.
2. Near the minimum the gradient is small, so the **steps shrink**. At the minimum the gradient is 0.
3. **Learning rate α:** too small is slow; too large **overshoots or diverges**.
4. **Example:** $J = \theta^2$ with $\theta_0 = 4$ and α = 0.1 gives θ = 4 → 3.2 → 2.56 → 2.05 → … → 0.

Batch GD	Stochastic GD	Mini-batch GD
Uses all data per step	One sample per step	Small batch per step
Stable but slow	Fast but noisy	**Best of both**

Conclusion: gradient descent trains regression models and every neural network (with Adam or SGD).

UNIT 3: DECISION TREES & INSTANCE-BASED LEARNING

Section A: 2-Mark Questions

Q1. Define Entropy; concept of instance-based learning. *[2 Marks]*

Ans.

- **Entropy** is the impurity of a set: $\text{Entropy}(S) = -\sum p_i \log_2 p_i$. It is 0 when the set is pure and 1 for a 50:50 split. Example: [9+, 5−] gives 0.940.
- **Instance-based (lazy) learning stores the training examples** and builds no model. At query time it predicts from the **most similar stored instances**. Example: **k-NN**.

Section B: 7-Mark Questions

Q2. Trace ID3 on the PlayTennis dataset. *[7 Marks]*

Ans. ID3 picks the attribute with the **highest Information Gain** at each node:

$$\text{Gain}(S, A) = \text{Entropy}(S) - \sum_v \frac{|S_v|}{|S|} \text{Entropy}(S_v)$$

Step 1: with S = [9+, 5−]:

$$\text{Entropy}(S) = -\frac{9}{14}\log_2\frac{9}{14} - \frac{5}{14}\log_2\frac{5}{14} = 0.940$$

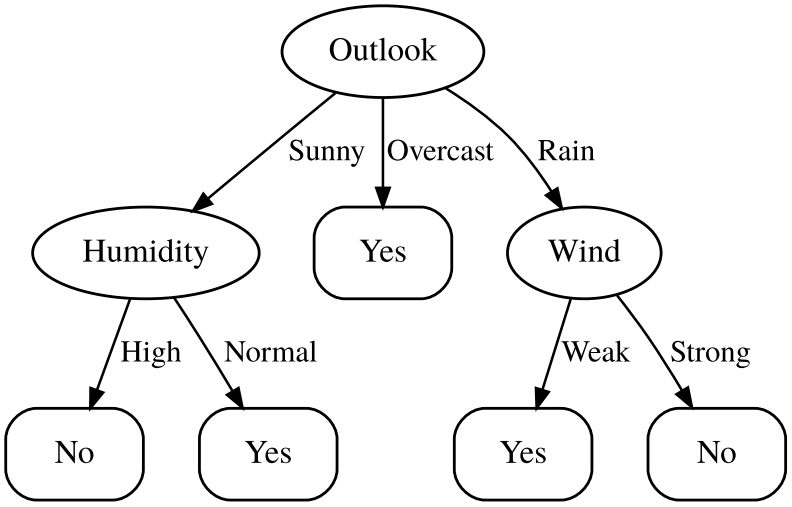
Step 2: Gain at the root

Attribute	Split [+, −]	Gain
Outlook	Sunny [2,3] 0.971, Overcast [4,0] 0, Rain [3,2] 0.971	0.940 − 0.694 = **0.246**
Humidity	High [3,4] 0.985, Normal [6,1] 0.592	0.151
Wind	Weak [6,2] 0.811, Strong [3,3] 1.0	0.048
Temperature	Hot [2,2], Mild [4,2], Cool [3,1]	0.029

Root = Outlook. Overcast is pure, so it becomes the leaf **Yes**.

Step 3: Sunny branch [2+, 3−]. Gain(Humidity) = **0.971** (both subsets pure), Temp = 0.571, Wind = 0.020. So **Humidity** is chosen.

Step 4: Rain branch [3+, 2−]. Gain(Wind) = **0.971**, Humidity = Temp = 0.020. So **Wind** is chosen.



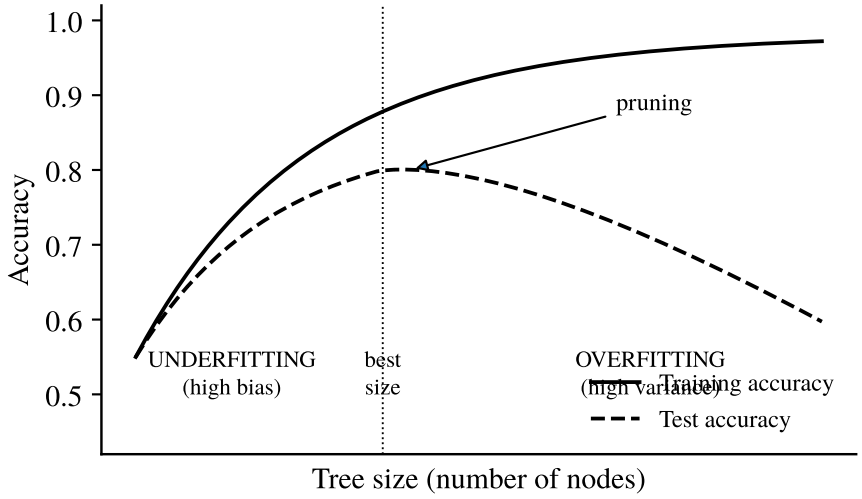
Final decision tree for PlayTennis

Conclusion: Temperature is never used. ID3 gives readable rules.

Q3. Overfitting and underfitting in decision trees; tree pruning. *[7 Marks]*

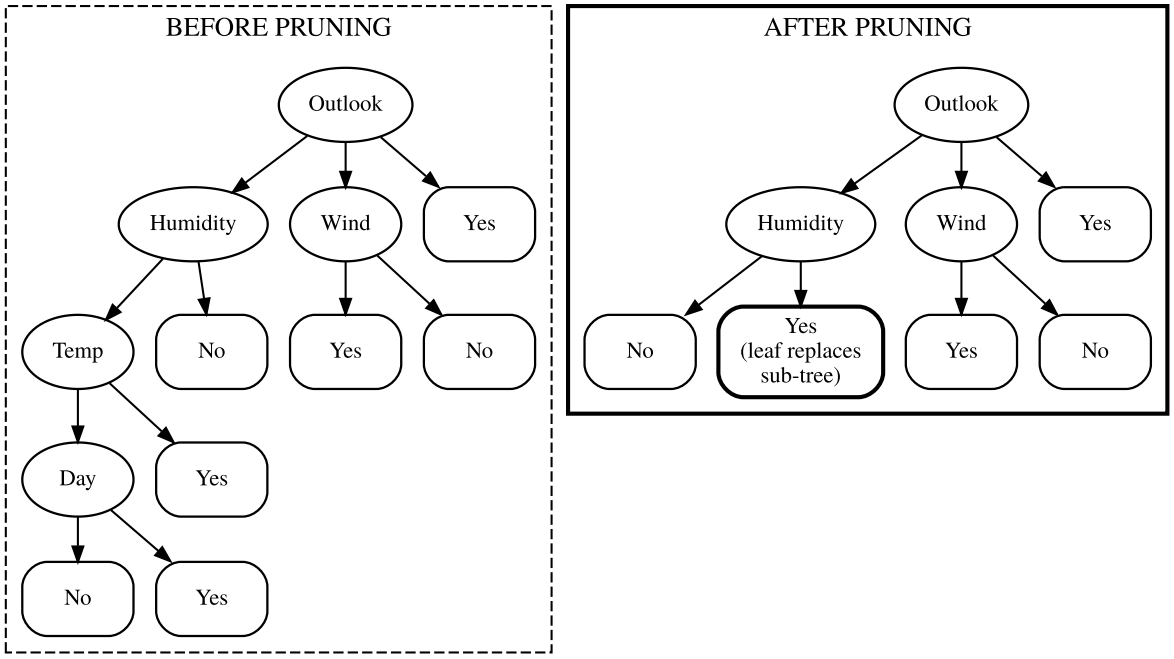
Ans.

- **Overfitting:** the tree is **too deep** and memorizes **noise**. Training accuracy is high but **test accuracy is low**.
- **Underfitting:** the tree is **too shallow**, so accuracy is low on both.
- **Pruning** removes branches that do not help generalization.



Accuracy vs. tree size: training accuracy keeps rising while test accuracy peaks and then falls (overfitting); pruning moves the tree back to the best size

1. **Causes of overfitting:**
   - Noisy data, too few examples.
   - Growing the tree until every leaf is pure.
   - Irrelevant attributes.
2. **Pre-pruning (early stopping):** stop at a max depth, a minimum number of samples, a gain below a threshold, or a chi-square test.
3. **Post-pruning:** grow the full tree, then prune it. This works better.
   - **Reduced Error Pruning:** replace a sub-tree with a leaf if **validation accuracy does not drop**.
   - **Rule post-pruning (C4.5):** convert the tree into rules and drop preconditions.
   - **Cost-complexity pruning (CART):** minimize $R(T) + \alpha|T|$.

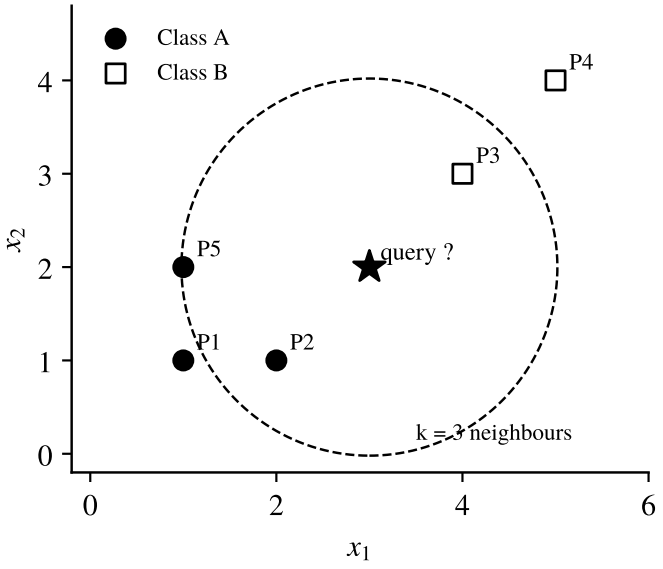


Before and after pruning: the noisy sub-tree under Humidity = Normal is replaced by its majority leaf

Underfitting	Overfitting
High bias	High variance
Low train and test accuracy	High train, low test accuracy
Fix: deeper tree	Fix: **pruning**, more data, Random Forest

Q4. Explain the k-NN algorithm and its limitations. *[7 Marks]*

Ans. k-NN is a **lazy, instance-based** classifier. It finds the **k nearest training points** and takes a **majority vote**.



k-NN with k = 3: the three nearest neighbours of the query (?) are A, A and B, so the majority class is A

1. Store all the training data.
2. Compute the distance to every point: $d = \sqrt{\sum \left(x_i - y_i\right)^2}$ (Euclidean).
3. Pick the k nearest.
4. Take a majority vote (or the mean, for regression).

Example: classify Q(3, 2) with k = 3, using P1(1,1) A, P2(2,1) A, P3(4,3) B, P4(5,4) B, P5(1,2) A.

- Distances: P2 = 1.41, P3 = 1.41, P5 = 2.0, P1 = 2.24, P4 = 2.83.
- The nearest 3 are A, B, A.

$$\text{Class} = A$$

Limitations:

1. **Slow prediction:** $O(n)$ distances per query.
2. Must **store all the data**.
3. **Curse of dimensionality.**
4. Needs **feature scaling**.
5. **Sensitive to noise** and irrelevant features.
6. The choice of k matters (use an odd k, chosen by cross-validation).

Q5. K-Means: cluster (185,72), (170,56), (168,60), (179,68), (182,72), (188,77) with K = 2.

[7 Marks]

Ans. Start with C1 = (185, 72) and C2 = (170, 56). Use $d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$.

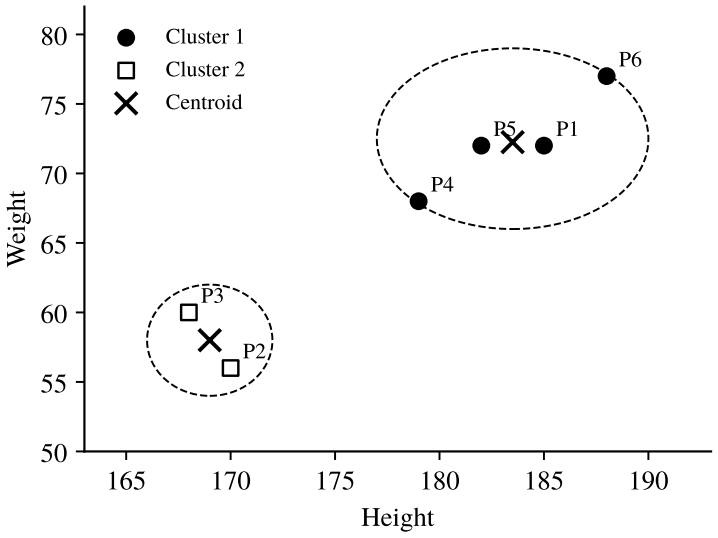
Iteration 1

Point	d to C1	d to C2	Cluster
(168,60)	$\sqrt{17^2 + 12^2} = 20.81$	$\sqrt{2^2 + 4^2} = 4.47$	C2
(179,68)	$\sqrt{6^2 + 4^2} = 7.21$	$\sqrt{9^2 + 12^2} = 15.0$	C1
(182,72)	3.00	20.00	C1
(188,77)	5.83	27.66	C1

New centroids:

$$C_1 = \left(\frac{185 + 179 + 182 + 188}{4}, \frac{72 + 68 + 72 + 77}{4}\right) = (183.5, 72.25), \qquad C_2 = (169, 58)$$

Iteration 2: every point keeps its cluster, so the algorithm has **converged**.



Final clusters after convergence (x = centroid)

$$C_1 = \{P1, P4, P5, P6\} \text{ at } (183.5, 72.25), \qquad C_2 = \{P2, P3\} \text{ at } (169, 58)$$

Q6. Compare Information Gain, Gain Ratio and Gini Index.

[7 Marks]

Ans. These are **attribute selection measures** for decision trees.

$$\text{GainRatio} = \frac{\text{Gain}}{\text{SplitInfo}}, \quad \text{SplitInfo} = -\sum \frac{|S_v|}{|S|} \log_2 \frac{|S_v|}{|S|}, \quad \text{Gini} = 1 - \sum p_i^2$$

Worked on Outlook (PlayTennis):

- Gain = 0.246.
- SplitInfo = 1.577, so GainRatio = 0.246 / 1.577 = **0.156**.
- Gini(S) = 0.459 and Gini_Outlook = 0.343, so the Gini gain = **0.116**.

Basis	Information Gain	Gain Ratio	Gini Index
Algorithm	**ID3**	**C4.5**	**CART**
Choose	Maximum	Maximum	**Minimum**
Bias	Many-valued attributes	Unbalanced splits	Larger partitions
Logarithm used?	Yes	Yes	**No (fastest)**
Split type	Multi-way	Multi-way	**Binary**

UNIT 4: ARTIFICIAL NEURAL NETWORKS

Section A: 2-Mark Questions

Q1. What is backpropagation? Inductive bias in ANN? *[2 Marks]*

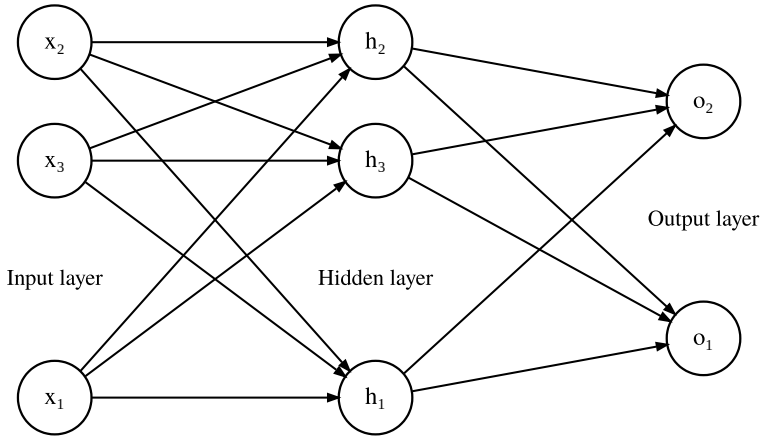
Ans.

- **Backpropagation** trains multilayer networks. A **forward pass** computes the outputs, then a **backward pass** sends the error δ from the output back to the hidden layers using the **chain rule**. Weights are updated by $\Delta w = \eta\, \delta\, x$.
- **Inductive bias of ANNs: smooth interpolation between data points**. The network architecture also adds bias (CNNs assume spatial locality).

Section B: 7-Mark Questions

Q2. Derive the backpropagation algorithm. *[7 Marks]*

Ans. Minimize $E = \frac{1}{2} \sum_k \left(t_k - o_k\right)^2$ by gradient descent, $\Delta w_{ji} = -\eta \frac{\partial E}{\partial w_{ji}}$, with sigmoid units $o = \sigma\left(net\right)$.



Forward pass computes the outputs; the backward pass sends the error terms δ from the output layer back to the hidden layer

Step 1: Chain rule.

$$\frac{\partial E}{\partial w_{ji}} = \frac{\partial E}{\partial net_j} \cdot x_{ji}$$

Step 2: Sigmoid derivative.

$$\frac{\partial o_j}{\partial net_j} = o_j\left(1 - o_j\right)$$

Step 3: Output unit.

$$\frac{\partial E}{\partial net_j} = \frac{\partial E}{\partial o_j} \cdot \frac{\partial o_j}{\partial net_j} = -\left(t_j - o_j\right) o_j\left(1 - o_j\right)$$

$$\delta_k = o_k\left(1 - o_k\right)\left(t_k - o_k\right)$$

Step 4: Hidden unit. It affects E through its downstream units k:

$$\frac{\partial E}{\partial net_h} = \sum_k \frac{\partial E}{\partial net_k} \cdot \frac{\partial net_k}{\partial o_h} \cdot \frac{\partial o_h}{\partial net_h} = \sum_k \left(-\delta_k\right) w_{kh}\, o_h\left(1 - o_h\right)$$

$$\delta_h = o_h\left(1 - o_h\right) \sum_k w_{kh}\, \delta_k$$

Step 5: Update.

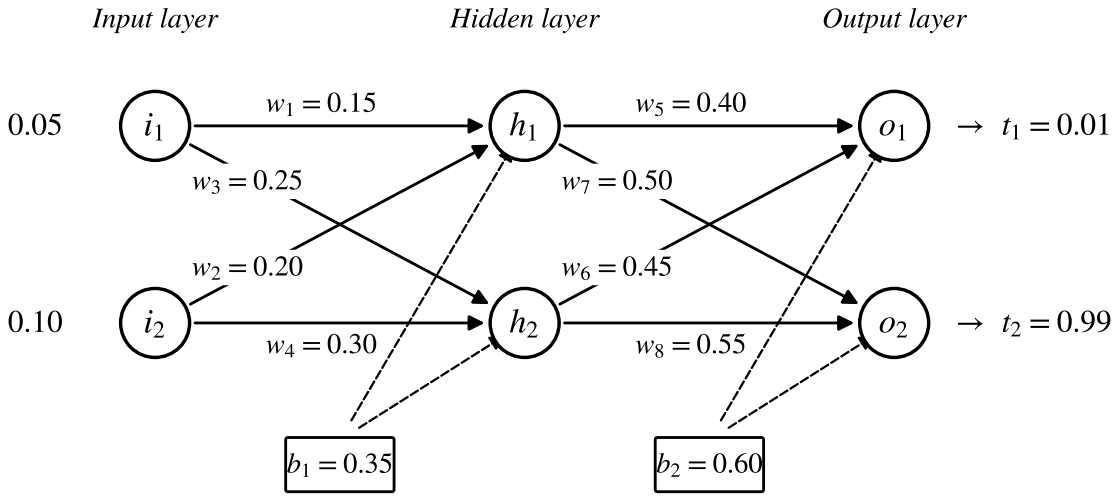
$$w_{ji} \leftarrow w_{ji} + \eta\, \delta_j\, x_{ji}$$

Algorithm: initialize small random weights; then repeat for each example: forward pass, compute δ for the outputs, compute δ for the hidden units, update all weights.

> **Remember:** the hidden δ is the **weighted sum of the downstream δs**. That is what "back-propagation" means.

Q3. Numerical: one forward and backward pass with η = 0.5. *[7 Marks]*

Ans.



Network with initial weights, biases and targets

Forward pass:

- $net_{h_1} = 0.15\,(0.05) + 0.20\,(0.10) + 0.35 = 0.3775$, so $out_{h_1} = \sigma\,(0.3775) = 0.5933$, and $out_{h_2} = 0.5969$.
- $net_{o_1} = 0.40\,(0.5933) + 0.45\,(0.5969) + 0.60 = 1.1059$, so $out_{o_1} = 0.7514$, and $out_{o_2} = 0.7729$.

$$E = \tfrac{1}{2}\,(0.01 - 0.7514)^2 + \tfrac{1}{2}\,(0.99 - 0.7729)^2 = 0.2984$$

Backward pass (output layer):

$$\delta_{o_1} = (0.01 - 0.7514)(0.7514)(0.2486) = -0.1385, \qquad \delta_{o_2} = +0.0381$$

- $w_5 = 0.40 + 0.5(-0.1385)(0.5933) = 0.3589$, $w_6 = 0.4087$, $w_7 = 0.5113$, $w_8 = 0.5614$.

Backward pass (hidden layer, using the OLD w5 to w8):

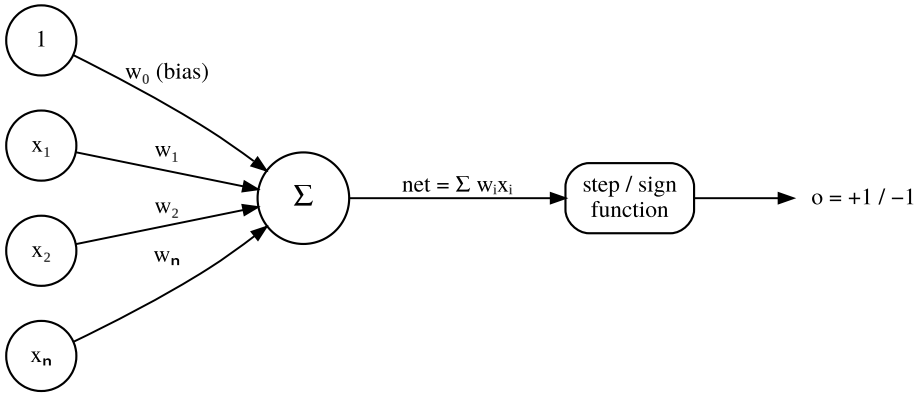
$$\delta_{h_1} = 0.2413\,[0.40(-0.1385) + 0.50(0.0381)] = -0.00877, \qquad \delta_{h_2} = -0.00995$$

- $w_1 = 0.15 + 0.5(-0.00877)(0.05) = 0.14978$, $w_2 = 0.19956$, $w_3 = 0.24975$, $w_4 = 0.29950$.

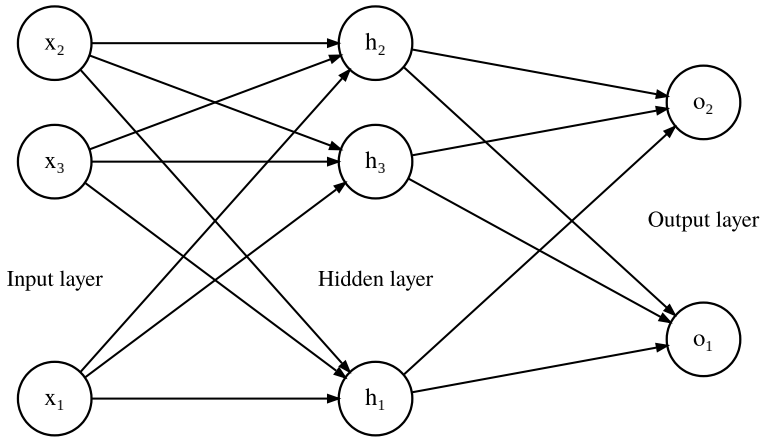
w	w1	w2	w3	w4	w5	w6	w7	w8
New	0.14978	0.19956	0.24975	0.29950	0.3589	0.4087	0.5113	0.5614

Q4. Single-layer perceptron vs. multilayer perceptron. *[7 Marks]*

Ans. An **SLP** has one layer of weights and a **step activation**, so it learns only **linearly separable** functions. An **MLP** has **hidden layers** with non-linear activations and is trained by **backpropagation**.

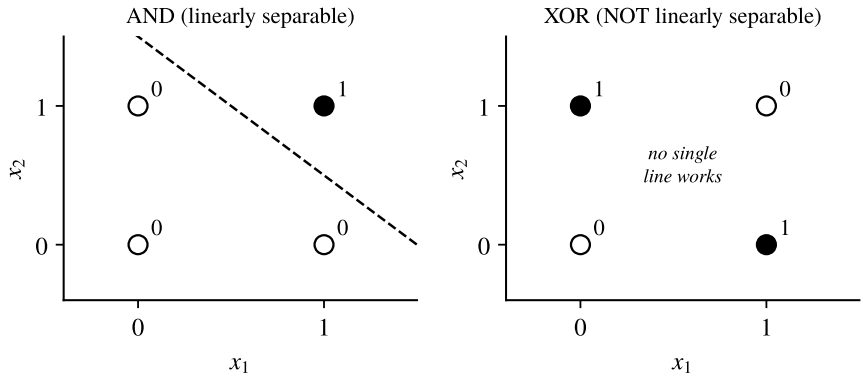


Single-layer perceptron: weighted sum followed by a step function



Multilayer perceptron with one hidden layer (sigmoid / ReLU units), trained by backpropagation

- **Perceptron rule:** $w_i \leftarrow w_i + \eta\,(t - o)\,x_i$. It converges if the data is linearly separable.
- **An SLP cannot learn XOR** (Minsky and Papert, 1969). An MLP can.



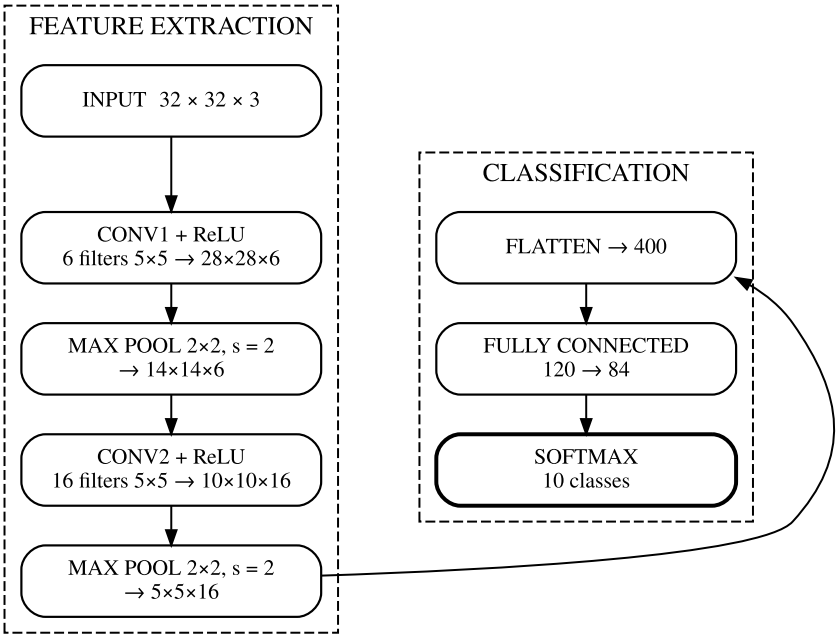
AND is linearly separable (one line separates the classes); XOR is not, so a single-layer perceptron cannot learn it

Basis	SLP	MLP
Hidden layer	None	One or more
Activation	Step	Sigmoid / ReLU
Learning	Perceptron rule	Backpropagation
Boundary	Linear	Non-linear
XOR	**No**	**Yes**

Q5. Explain the CNN layered architecture.

[7 Marks]

Ans. A **CNN** learns **spatial features** from images using **convolution, pooling and fully connected** layers.



CNN architecture (LeNet-style): feature extraction (left), then classification (right)

1. **Convolution:** a filter slides over the input and takes **dot products**, producing **feature maps**. It uses **weight sharing** and local connectivity. Output size:

$$\frac{W - F + 2P}{S} + 1 \qquad \text{for example } \frac{32 - 5}{1} + 1 = 28$$

2. **ReLU:** $\max(0, x)$ adds non-linearity.
3. **Pooling:** down-samples, has **no learnable weights**, and gives translation invariance. Example:

$$\begin{bmatrix} 1 & 3 & 2 & 4 \\ 5 & 6 & 1 & 2 \\ 7 & 2 & 9 & 1 \\ 3 & 4 & 0 & 8 \end{bmatrix} \xrightarrow{\max\ 2\times 2} \begin{bmatrix} 6 & 4 \\ 7 & 9 \end{bmatrix}$$

4. **Flatten → Fully connected → Softmax:** $P_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$.

Convolution	Pooling	Fully Connected
Extracts features	Down-samples	Classifies
Has learnable weights	No weights	Most of the weights

Conclusion: used for face recognition, medical imaging and self-driving cars.

UNIT 5: REINFORCEMENT LEARNING & GENETIC ALGORITHMS

Section A: 2-Mark Questions

Q1. What is Q-learning? Crowding in GA? *[2 Marks]*

Ans.

- **Q-learning** is **model-free, off-policy** RL. It learns $Q(s, a)$ with:

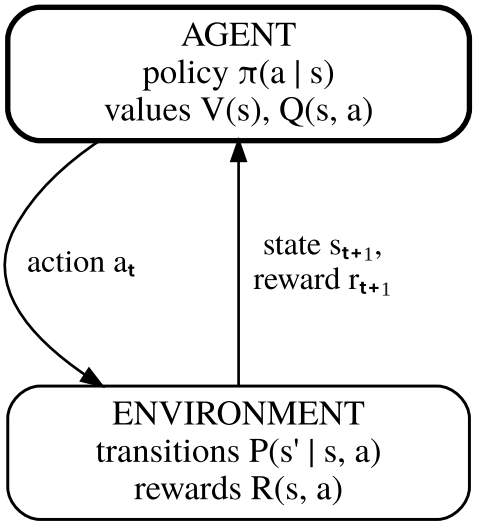
$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s,a)]$$

- **Crowding:** a very fit individual and its copies **take over the population**. Diversity is lost and the GA **converges prematurely**. Fixes: **fitness sharing**, crowding replacement, tournament selection.

Section B: 7-Mark Questions

Q2. Explain RL through the Markov Decision Process. *[7 Marks]*

Ans. RL is modelled as an **MDP (S, A, P, R, γ)**. An agent chooses actions to **maximize the discounted return** $G_t = \sum_k \gamma^k r_{t+k+1}$.

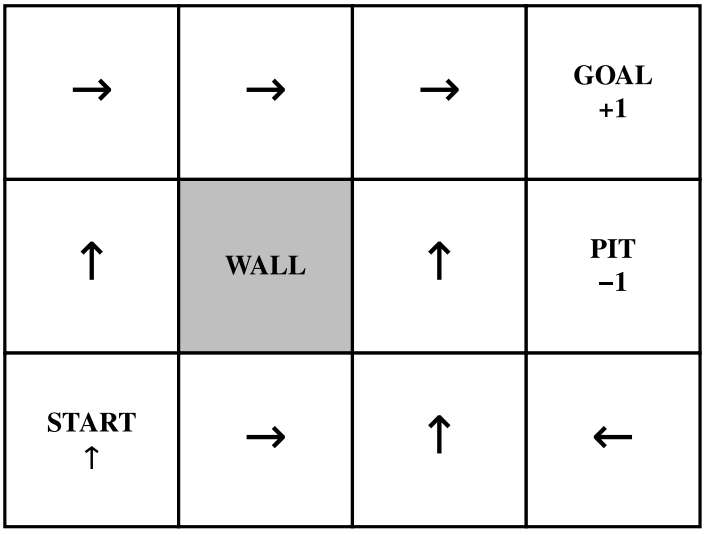


Agent-environment interaction in an MDP

1. **S, A:** the states and actions.
2. **P(s' | s, a):** the transition probability.
3. **R:** the reward.
4. **γ:** the discount factor.
5. **Markov property:** the next state depends **only on the current state and action**.
6. **Policy π(s):** maps states to actions. **Value:** $V^{\pi}(s) = E\left[\sum \gamma^k r\right]$.
7. **Bellman optimality equation:**

$$V^{*}(s) = \max_{a} \sum_{s'} P\left(s'|s,a\right)\left[R + \gamma V^{*}\left(s'\right)\right]$$

8. **Solving it: value iteration or policy iteration** (when the model is known), **Q-learning or SARSA** (when it is not). Use **ε-greedy** exploration.

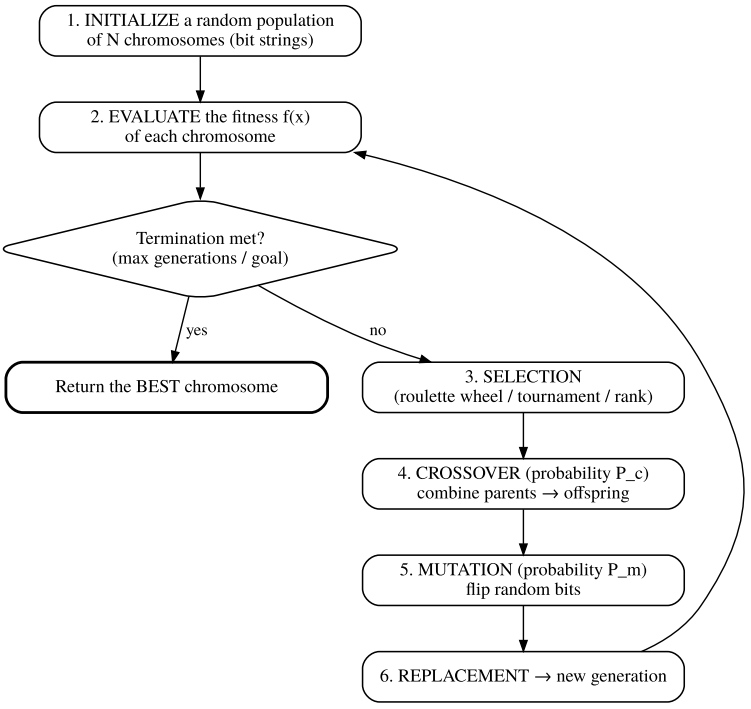


Grid-world MDP ($\gamma = 0.9$; +1 at GOAL, −1 at PIT, −0.04 per step; 0.8 chance of the intended move). Arrows show the optimal policy π^*

Conclusion: used in AlphaGo, robotics and RLHF.

Q3. Trace the Genetic Algorithm cycle (fitness, crossover, mutation). *[7 Marks]*

Ans. A **GA** evolves a population of chromosomes through **selection, crossover and mutation**.



Genetic Algorithm cycle

Trace: maximize $f(x) = x^2$ for x in 0 to 31 (5 bits).

Chromosome	x	f(x)	P(select)
01101	13	169	0.14
11000	24	576	**0.49**
01000	8	64	0.06
10011	19	361	0.31

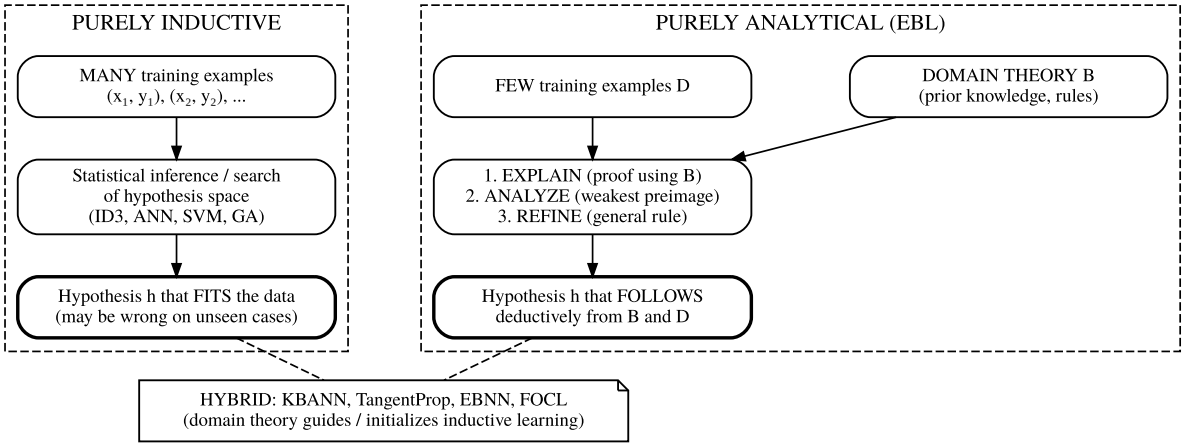
- Sum = 1170, average = 293, max = 576.
- **Crossover:**
  - 0110|1 × 1100|0 gives 01100 (144) and 11001 (625).
  - 11|000 × 10|011 gives **11011 (729)** and 10000 (256).
- After one generation: **average 293 → 439, max 576 → 729**.
- **Mutation:** flip a random bit with small probability P_m (for example 10000 → 10100). This keeps diversity.

Roles:

- **Fitness:** measures quality and guides selection.
- **Crossover: exploitation** (combines good building blocks).
- **Mutation: exploration** (prevents premature convergence).

Q4. Compare purely analytical and purely inductive learning. *[7 Marks]*

Ans. Inductive learning generalizes from **many examples**, with no prior knowledge. **Analytical learning (EBL)** uses a **domain theory plus a few examples** and **explains** them deductively.



Purely inductive vs. purely analytical learning

Basis	Inductive	Analytical
Input	Many examples	Domain theory + few examples
Reasoning	**Induction** (statistical)	**Deduction** (logical proof)
Prior knowledge	Not needed	**Essential**
Risk	Too little data gives wrong hypotheses	**Wrong domain theory** gives wrong hypotheses
Examples	ID3, ANN, SVM	**Prolog-EBG**

EBL steps:

1. **Explain** the example using the theory.
2. **Analyze** it (weakest preimage).
3. **Refine** it into a general rule.

Hybrid methods: KBANN (builds a network from rules, then trains it).